

sort 정리

범위 안의 데이터를 정렬하는 가장 많이 쓰는 STL 알고리즘입니다.

사용처 (Where)

- 숫자, 문자열, pair, 구조체 목록을 정렬할 때
- 정렬 후 이분 탐색을 사용할 때
- 순위, 최소/최대 그룹을 다룰 때

방법 (How to Use)

- 정렬할 컨테이너의 begin/end를 넘김
- 기본은 오름차순
- greater<T>() 또는 lambda로 기준 변경
- vector, array, deque처럼 임의 접근 반복자가 필요

기본 정보

- 필요 헤더: #include <algorithm>
- 기본 선언: sort(v.begin(), v.end());

왜 써야 할까? (Why)

- 직접 정렬 구현보다 훨씬 짧고 안정적
- 대부분의 정렬 문제에서 충분히 빠름
- lambda와 함께 쓰면 기준을 자유롭게 바꿀 수 있음

주요 함수

함수/문법	의미
sort(first,last)	오름차순 정렬
sort(first,last,comp)	사용자 기준 정렬
greater<int>()	내림차순
stable_sort	같은 값의 기존 순서 유지

기본 코드 예시

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

int main() {
    vector<int> v = {3, 1, 4, 2};
    sort(v.begin(), v.end());

    for (int x : v) cout << x << ' ';
}
```

sort 비교와 응용

STL을 직접 구현이나 C 스타일 코드와 비교하고, 실제 문제에서 쓰는 방식을 확인합니다.

시간 복잡도

동작	복잡도
sort	$O(n \log n)$
비교 함수	매 비교마다 호출
list 정렬	list.sort() 사용

STL vs 직접 구현 vs C 스타일

구분	STL	직접 구현	C 스타일
개발 난이도	sort 함수 호출만 알면 됨	반복문과 조건을 직접 설계	qsort/직접 반복문 사용
코드 길이	짧고 의도가 분명함	길어지고 실수 지점 증가	자료형 처리와 캐스팅이 번거로움
안정성	표준 라이브러리라 검증됨	구현 실수 가능	포인터/범위 실수 가능
추천 상황	대부분의 일반 상황	원리를 공부하거나 특수 최적화	C 코드와 연동할 때

응용: 구조체를 점수 내림차순으로 정렬

lambda 비교 함수를 사용해 원하는 기준을 직접 지정합니다.

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

struct Student {
    string name;
    int score;
};

int main() {
    vector<Student> v = {"Kim", 80}, {"Lee", 95}, {"Park", 90};
    sort(v.begin(), v.end(), [](const Student& a, const Student& b) {
        return a.score > b.score;
    });

    for (auto s : v) cout << s.name << ' ' << s.score << '\n';
}
```

처음 배울 때 자주 하는 실수

- list에는 sort(v.begin(), v.end())를 쓸 수 없음
- 비교 함수는 strict weak ordering을 지켜야 함
- 정렬하면 원래 순서가 바뀜

비슷한 항목과 차이

- `stable_sort`는 같은 값의 기존 순서를 유지
- `partial_sort`는 일부만 정렬
- `nth_element`는 k번째 기준 분할에 적합

한 줄 정리: `sort`은(는) '숫자, 문자열, `pair`, 구조체 목록을 정렬할 때' 상황에서 먼저 떠올리면 좋은 STL입니다.